

Reporte Técnico: Análisis de Sentimientos en Tweets de Aerolíneas

Comparativa de Arquitecturas Recurrentes: RNN, GRU y LSTM

1. Introducción y Objetivo

Este proyecto aborda la clasificación de sentimientos en tweets dirigidos a aerolíneas estadounidenses. El objetivo es entrenar y comparar tres arquitecturas de redes neuronales recurrentes, RNN, GRU y LSTM, para clasificar cada tweet en una de tres clases: negativo, neutral o positivo.

La tarea es relevante en el ámbito del NLP porque los tweets son secuencias de longitud variable, con lenguaje informal, abreviaciones y ruido, lo que convierte esta tarea en un proceso complejo..

2. Dataset y Carga de Datos

2.1 Descripción general

El dataset utilizado es el conocido US Airline Twitter Sentiment con las siguientes características brutas:

Métrica	Valor
Total de tweets	14,640
Tweets tras limpieza	14,640 (sin nulos en columnas clave)
Número de clases	3 (negative, neutral, positive)
Tamaño de vocabulario	6,106 tokens

El dataset contiene 15 columnas originales, incluyendo: tweet_id, airline_sentiment, airline_sentiment_confidence, negativereason, negativereason_confidence, airline,

airline_sentiment_gold, name, negativereason_gold, retweet_count, text, tweet_coord, tweet_created, tweet_location, user_timezone.

2.2 Distribución de clases (desbalance)

Este desbalance severo (la clase negativa triplica a las demás) es uno de los retos iniciales del proyecto. Si no se trata, los modelos tienden a predecir casi exclusivamente la clase mayoritaria, obteniendo un accuracy aparentemente alto pero un F1 muy bajo en las clases minoritarias.

```
# Total de tweets: 14640
# Clases: {'negative': 9178, 'neutral': 3099, 'positive': 2363}
```

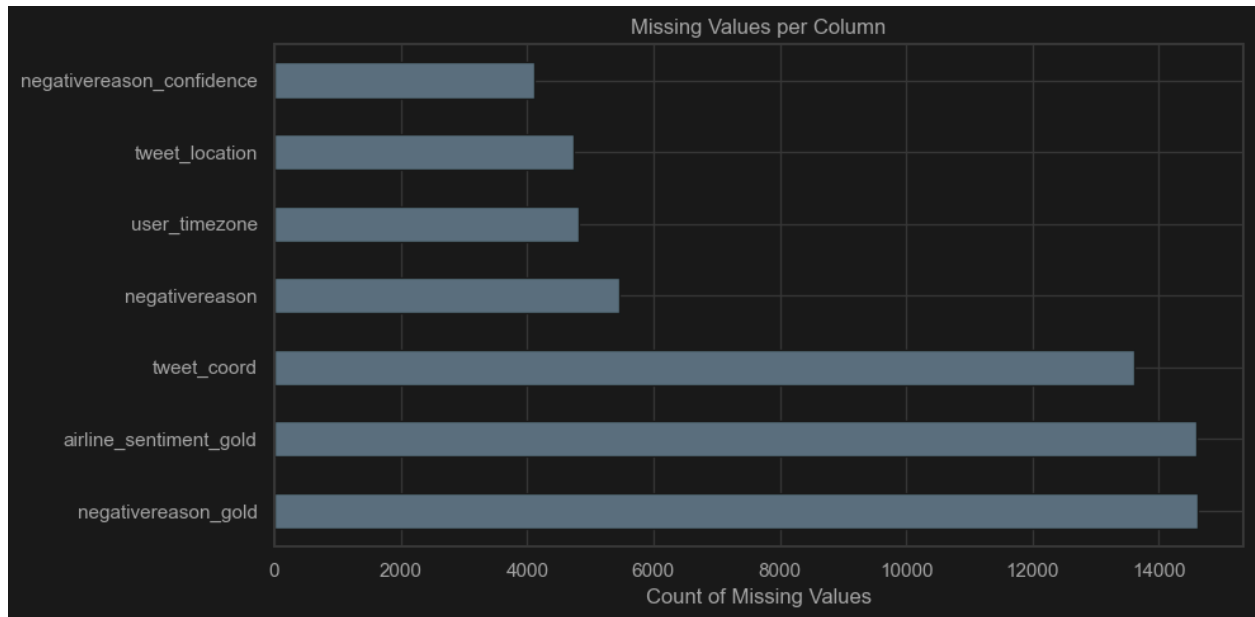
3. Análisis Exploratorio de Datos (EDA)

3.1 Valores faltantes

El gráfico de Missing Values per Column revela que varias columnas presentan una cantidad crítica de nulos:

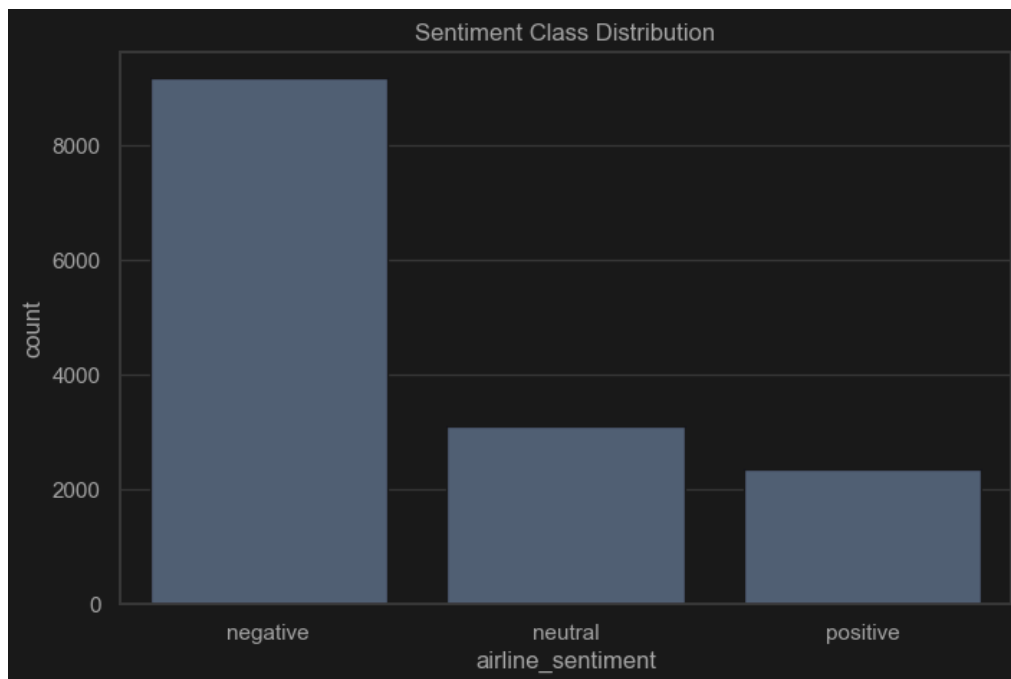
- negativereason_gold y airline_sentiment_gold: ~14,500 nulos cada una (prácticamente vacías, son anotaciones alternativas que no se usan).
- tweet_coord: ~13,500 nulos — la gran mayoría de tweets no incluye coordenadas geográficas.
- negativereason, user_timezone, tweet_location: entre 4,000 y 5,500 nulos.

Para el flujo de modelado solo se necesitan las columnas text y airline_sentiment, que no presentan nulos, por lo que el preprocesamiento no requiere imputación.



3.2 Distribución de clases

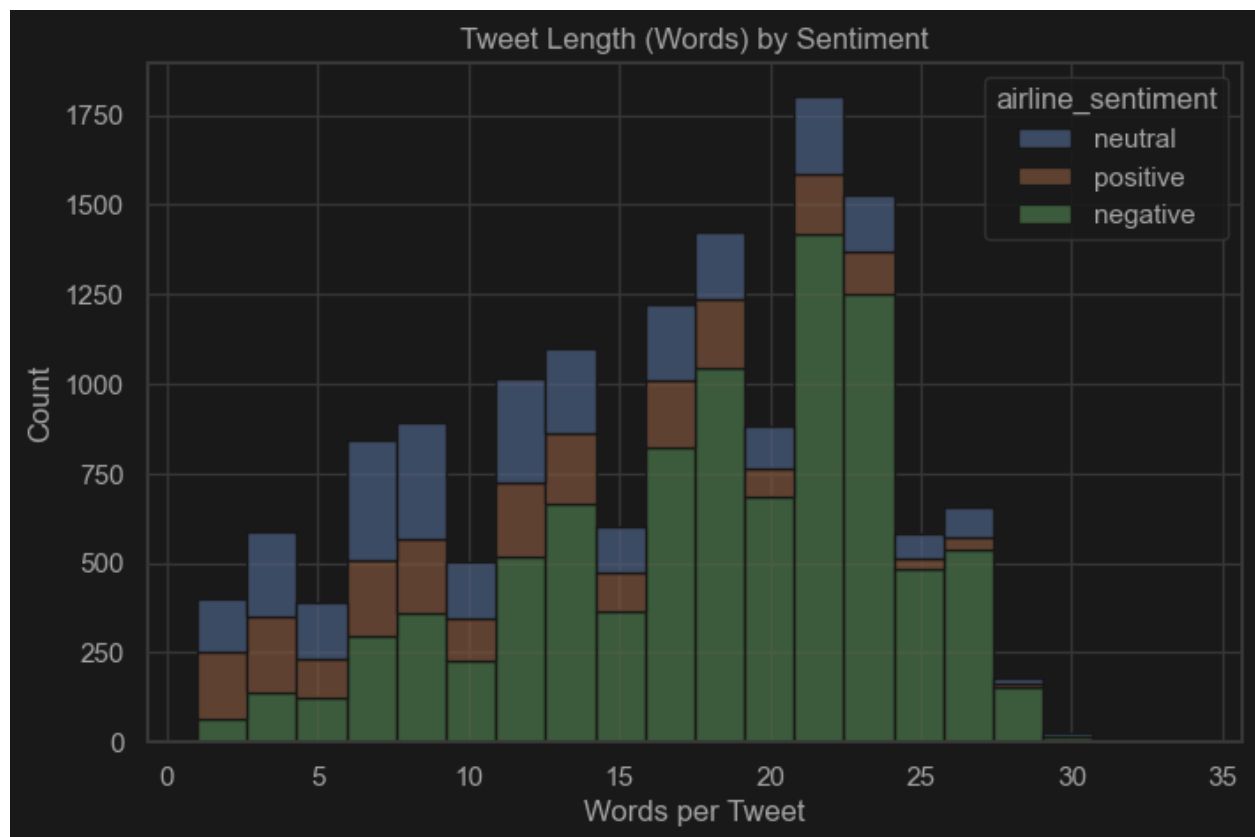
El gráfico Sentiment Class Distribution confirma el desequilibrio: la barra de *negative* (~9,178) es aproximadamente 3 veces mayor que *neutral* (~3,099) y casi 4 veces mayor que *positive* (~2,363). Este patrón es natural, dado que los usuarios recurren a Twitter principalmente para quejarse, lo que infla la clase negativa.



3.3 Longitud de secuencias por sentimiento

El histograma Tweet Length (Words) by Sentiment muestra que:

- La mayoría de tweets varían entre 10 y 25 palabras, con un pico alrededor de las 20-22 palabras.
- Las tres clases de sentimiento tienen distribuciones de longitud similares, lo que indica que la longitud del texto no es en sí misma un predictor fuerte del sentimiento.
- La clase negativa domina en todos los rangos de longitud, coherente con su mayor frecuencia global.
- La elección de MAX_SEQ_LENGTH = 50 captura prácticamente todas las secuencias sin truncado excesivo.

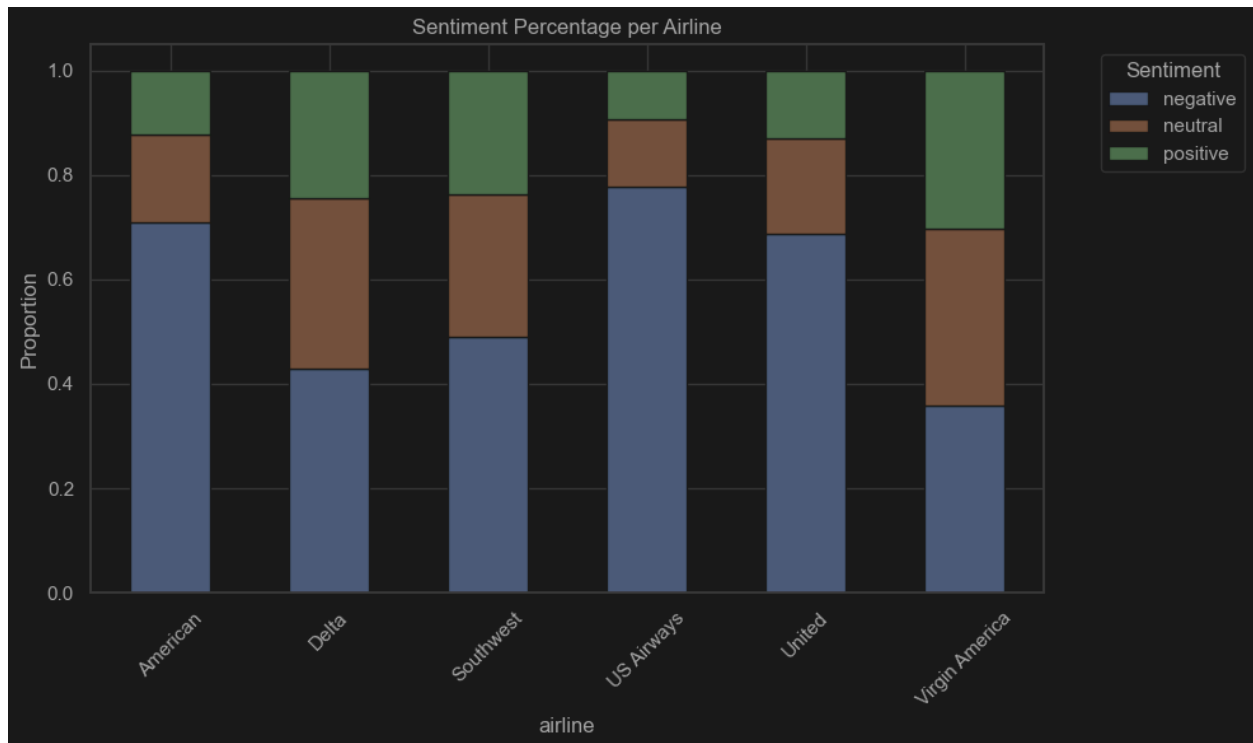


3.4 Sentimiento por aerolínea

El gráfico Sentiment Percentage per Airline es uno de los más informativos del EDA:

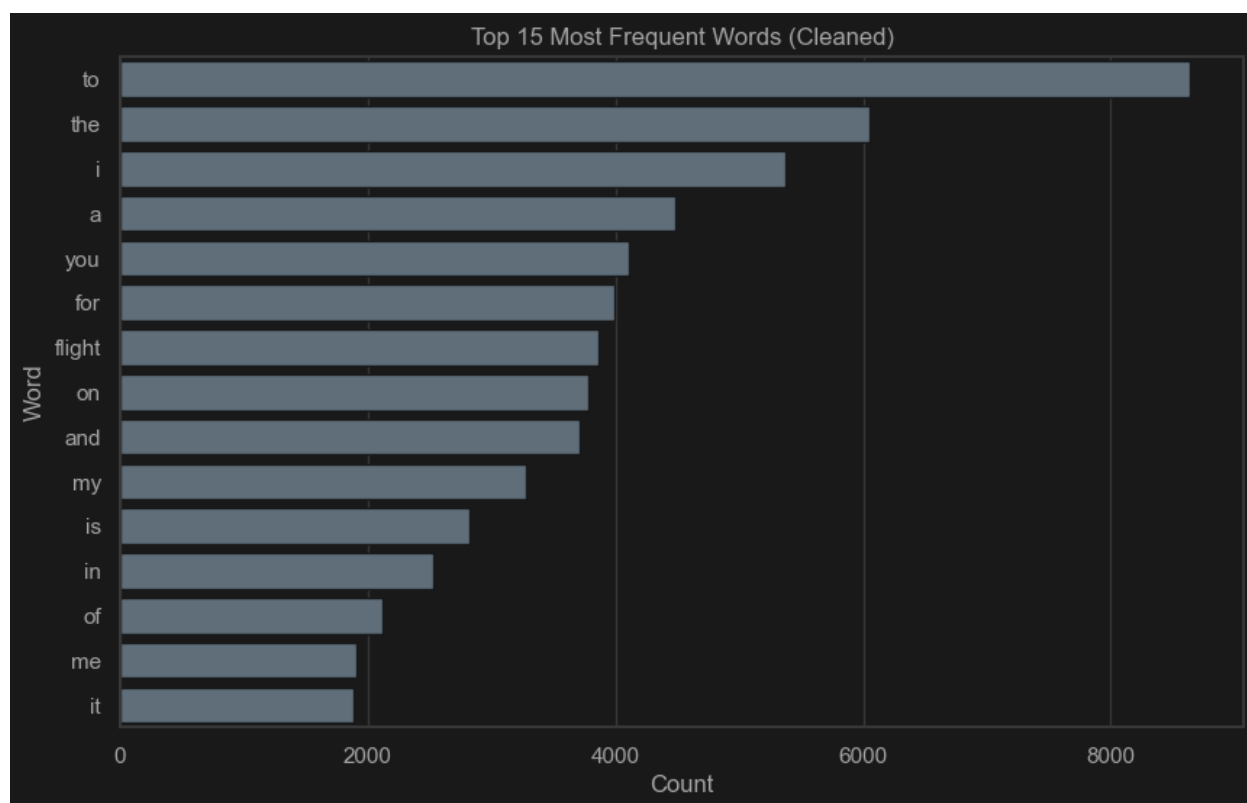
- US Airways y United presentan las proporciones más altas de sentimiento negativo (78% aprox. y 69% aprox. respectivamente), sugiriendo que fueron las aerolíneas con peor percepción pública en el período de captura de datos.

- Virgin America destaca positivamente con la menor proporción de negativos (35%) y la mayor de positivos (30%), siendo la aerolínea con mejor imagen en Twitter.
- Delta y Southwest muestran un perfil más equilibrado, con proporciones intermedias de negativo (42-49%) y mayor presencia de neutral que el resto.



3.5 Palabras más frecuentes

El gráfico Top 15 Most Frequent Words (Cleaned) muestra que sin eliminar stopwords, las palabras más comunes son partículas gramaticales: *to*, *the*, *i*, *a*, *you*, *for*. Esto refuerza la importancia de aplicar filtrado de stopwords antes del análisis léxico. La primera palabra con contenido semántico real es *flight* (posición 7), lo que confirma la temática del análisis. Un análisis post-stopwords revelaría términos como *cancelled*, *delayed*, *service*, *help* como los más discriminativos.



4. Preprocesamiento y Pipeline de Texto

Lo primero que hice fue eliminar las stopwords, excepto las de negación, ya que una frase como: “not so great” es negativa pero por el simple hecho de haber un great el modelo entiende que es positiva. Vi también que las stopwords no afectaban tanto para las reseñas con sentimientos positivos o neutros.

4.1 Limpieza de texto (clean_text)

La función aplica las siguientes transformaciones en orden:

1. Conversión a minúsculas.
2. Eliminación de URLs (<http://>, www., <https://>).
3. Eliminación de menciones (@usuario) y hashtags (#tema).
4. Eliminación de caracteres especiales (solo conserva alfanuméricos y espacios).
5. Normalización de espacios múltiples.

El resultado es texto limpio, sin ruido propio de Twitter, listo para tokenización.

4.2 Construcción del vocabulario (build_vocab)

La función construye un diccionario que mapea cada palabra a un índice numérico, esencial porque las redes neuronales operan con números, no con texto.

El proceso es:

1. Conteo de frecuencias de todas las palabras en el corpus de entrenamiento.
2. Filtrado: solo se incluyen palabras que aparecen al menos `MIN_WORD_FREQ = 2` veces. Esto elimina erratas y términos rarísimos que añadirían ruido.
3. Asignación de índices a partir del 2, reservando los índices especiales:
 - `<PAD>` → índice 0: token de relleno para igualar longitudes.
 - `<UNK>` → índice 1: representa palabras no vistas en entrenamiento.

Resultado: vocabulario de 6,106 tokens.

Ejemplo práctico con `MIN_WORD_FREQ = 2`:

"great flight very happy" → great:3, flight:2, very:1, happy:1

Vocab resultante: {0, 1, great:2, flight:3}

"great flight very happy" → [2, 3, 1, 1] (very y happy → UNK)

4.3 Tokens especiales: `<PAD>` y `<UNK>`

`<PAD>` (Padding): Las redes neuronales requieren inputs de longitud fija. Tweets cortos se rellenan con ceros hasta alcanzar `MAX_SEQ_LENGTH = 50`:

`<UNK>` (Unknown): Palabras que no están en el vocabulario (por baja frecuencia o por ser del conjunto de test) se sustituyen por el índice 1. Esto evita errores de indexación y permite al modelo aprender una representación genérica de "palabra desconocida".

4.4 Conversión a secuencias (text_to_sequence)

Convierte texto limpio a una lista de índices enteros de longitud exactamente `MAX_SEQ_LENGTH`:

- Si el tweet es más corto: se añaden `<PAD>` al final.
- Si es más largo: se trunca a los primeros 50 tokens.

4.5 Codificación de etiquetas y splits

Las etiquetas se codifican con `LabelEncoder`:

negative: 0 | neutral: 1 | positive: 2

La partición estratificada garantiza la misma proporción de clases en cada split:

Train: 70% | Val: 15% | Test: 15%

5. Balanceo de Clases con SMOTE

5.1 Motivación

El desbalance 62.7% / 21.2% / 16.1% haría que los modelos ignorasen las clases minoritarias. Para corregirlo se aplica SMOTE (Synthetic Minority Over-sampling Technique) exclusivamente sobre el conjunto de entrenamiento (nunca sobre val o test, para evitar data leakage).

5.2 Resultado

Antes de SMOTE: 10,248 muestras (negative: ~6,425 | neutral: ~2,170 | positive: ~1,653)

Después de SMOTE: 19,275 muestras (negative: 6,425 | neutral: 6,425 | positive: 6,425)

SMOTE genera ejemplos sintéticos interpolando entre muestras reales de las clases minoritarias en el espacio de features (las secuencias numéricas). El resultado es un conjunto perfectamente balanceado, triplicando el tamaño efectivo del entrenamiento.

Consecuencia sobre los pesos: al estar perfectamente balanceado post-SMOTE, los pesos de clase calculados con `compute_class_weight` resultan todos iguales a 1.0, lo que es coherente: ya no hay necesidad de compensación adicional en la función de pérdida.

6. Arquitectura del Dataset Personalizado (TwitterDataset)

La clase `TwitterDataset` hereda de `torch.utils.data.Dataset` y actúa como interfaz entre los datos preprocesados y el pipeline de entrenamiento de PyTorch.

`__init__` → almacena textos, etiquetas y vocabulario

`__len__` → informa al `DataLoader` del número de ejemplos

`__getitem__` → convierte el tweet *i*-ésimo en (`tensor_secuencia`, `tensor_etiqueta`)

El `DataLoader` llama internamente a `__getitem__` para construir batches de 32 ejemplos, gestionando shuffle en train y carga eficiente en val/test.

7. Arquitecturas de Redes Neuronales

Los tres modelos comparten la misma estructura modular:

texto (índices) → Embedding → Dropout → [RNN|GRU|LSTM] → FC → logits (3 clases)

7.1 Capa de Embedding

```
nn.Embedding(vocab_size=6106, embedding_dim=100, padding_idx=0)
```

Convierte cada índice entero en un vector denso de 100 dimensiones. Estos vectores se aprenden durante el entrenamiento. El argumento `padding_idx=0` fija el embedding de <PAD> a cero y no lo actualiza en el backprop, evitando que el relleno contamine los gradientes.

Dimensiones: entrada (batch, 50) → salida (batch, 50, 100).

7.2 Dropout

```
nn.Dropout(p=0.3)
```

Durante el entrenamiento, desactiva aleatoriamente un porcentaje de las neuronas (determinado por Optuna) en cada forward pass. Durante la evaluación, no aplica ningún cambio. Su función es regularizar el modelo, forzando redundancia en las representaciones aprendidas y reduciendo el overfitting.

7.3 Capa Completamente Conectada (FC)

```
nn.Linear(hidden_dim, output_dim=3)
```

Recibe el último estado oculto de la red recurrente (cuya dimensión es definida por el hiperparámetro `hidden_dim` en Optuna) y lo proyecta a 3 dimensiones (una por clase). Los logits resultantes se usan directamente con `CrossEntropyLoss`/`FocalLoss`.

7.4 RNN (Recurrent Neural Network)

La arquitectura más simple. En cada paso de tiempo, el estado oculto se actualiza. Solo tiene una operación de recurrencia directa. Problema fundamental: el gradiente que fluye hacia atrás a través de muchos pasos de tiempo se multiplica repetidamente, tendiendo a cero (vanishing gradient) o a infinito (exploding gradient). En la práctica, la RNN tiene dificultades para capturar dependencias a más de 10-15 pasos de distancia. Los hiperparámetros como la dimensión

oculta (`hidden_dim`), el número de capas (`num_layers`) y el dropout son determinados dinámicamente mediante optimización con Optuna.

7.5 GRU (Gated Recurrent Unit)

Introduce dos puertas que controlan el flujo de información:

- Puerta de reset (r_t): decide cuánto del estado anterior olvidar al calcular el nuevo estado candidato.
- Puerta de update (z_t): decide cuánto del estado anterior conservar vs. cuánto del nuevo estado candidato incorporar.

Las puertas crean atajos para el gradiente, mitigando el vanishing gradient. GRU es más eficiente que LSTM (menos parámetros) con rendimiento comparable en muchas tareas.

También añadí bidireccionalidad para que leyera los tweets tanto de izquierda a derecha como de derecha a izquierda

7.6 LSTM (Long Short-Term Memory)

La arquitectura más compleja. Introduce un cell state (c_t) como memoria a largo plazo separada del estado oculto, controlado por tres puertas:

- Puerta de olvido (f_t): qué información del cell state anterior descartar.
- Puerta de entrada (i_t): qué nueva información escribir en el cell state.
- Puerta de salida (o_t): qué parte del cell state exponer como estado oculto.

La separación entre cell state y hidden state permite mantener información relevante a muy largo plazo sin que se degrade. Es la arquitectura más expresiva de las tres, a costa de mayor tiempo de cómputo.

Como en GRU, añadí bidireccionalidad.

8. Pipeline de Entrenamiento

8.1 Función de pérdida

Se usa FocalLoss (alternativa a CrossEntropyLoss) con $\gamma=2.0$:

$$\text{focal_loss} = (1 - p_t)^\gamma \times \text{ce_loss}$$

La Focal Loss reduce el peso de los ejemplos fáciles (bien clasificados, p_t alta) y focaliza el aprendizaje en los difíciles. Esto es especialmente útil cuando el SMOTE genera ejemplos sintéticos que pueden ser más fáciles de clasificar.

8.2 Optimizador

Adam. Combina momentum y adaptación de learning rate por parámetro, siendo el estándar de facto para redes recurrentes en NLP. La tasa de aprendizaje (learning rate) óptima es buscada activamente mediante Optuna en escala logarítmica.

8.3 Early Stopping

Se utiliza el MedianPruner de Optuna durante la fase de búsqueda para detener prematuramente los ensayos (trials) que no prometen buenos resultados, reduciendo los tiempos de entrenamiento innecesarios. En el entrenamiento final, se guarda continuamente el estado del modelo cada vez que se obtiene una mejor precisión (accuracy) en el conjunto de validación, garantizando que se restaure el modelo con mayor capacidad de generalización.

8.4 Optimización con Optuna

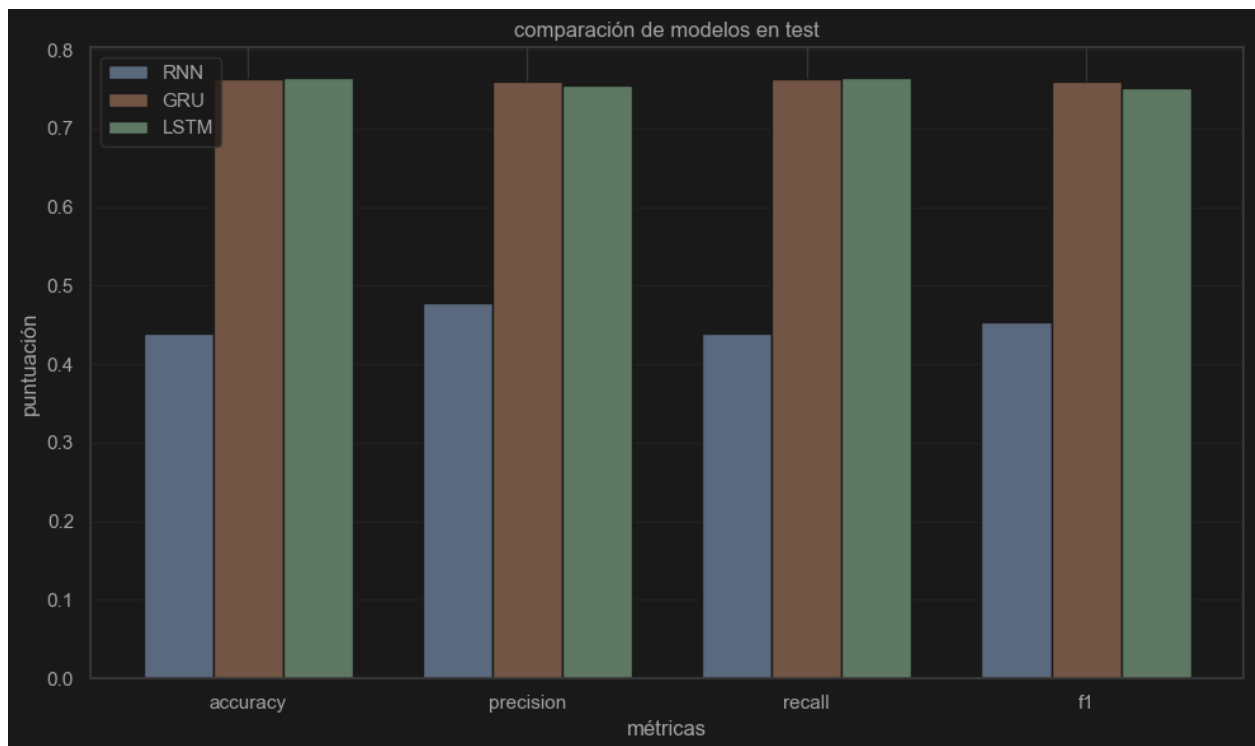
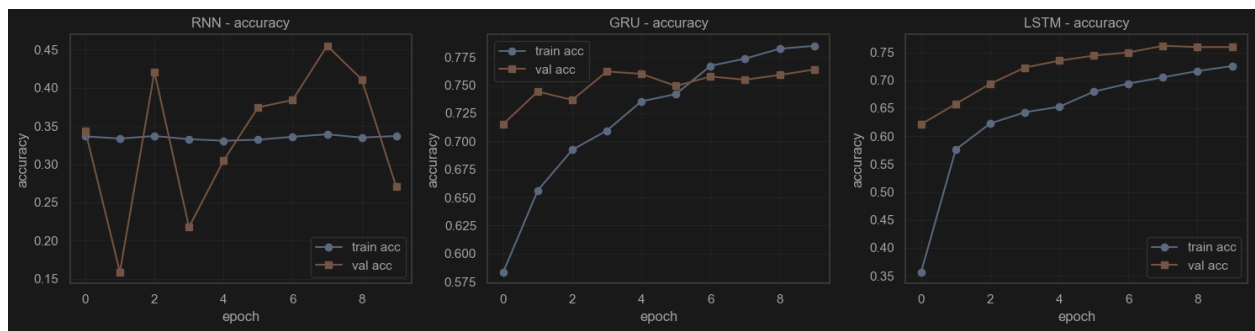
Para encontrar la configuración óptima de los modelos, se integró Optuna. El estudio realiza múltiples ensayos (trials) buscando maximizar el accuracy de validación. El espacio de búsqueda definido incluye:

- Arquitectura: RNN, GRU, LSTM
- Dimensión de Embedding (embedding_dim): 64, 128, 256
- Dimensión Oculta (hidden_dim): 64, 128, 256
- Número de Capas (num_layers): 1 a 3
- Dropout: 0.1 a 0.5
- Tasa de Aprendizaje (learning rate): $1e-4$ a $1e-2$ (escala logarítmica)
- Tamaño de Lote (batch_size): 64, 128

Tras realizar la optimización, se extraen los mejores parámetros y se procede al entrenamiento definitivo de los tres modelos (RNN, GRU y LSTM) empleando esta configuración óptima.

9. Resultados del Entrenamiento





9.1 RNN

Las curvas de loss muestran alta variabilidad —especialmente el val loss, que oscila de forma errática— y el train accuracy decrece en las últimas épocas (de 0.44 a 0.37), señal clara de inestabilidad en el entrenamiento. El modelo no logra aprender representaciones útiles de las secuencias. El early stopping actúa en la época 10.

9.2 GRU

Las curvas de GRU muestran un aprendizaje estable y progresivo. El train loss cae monotónicamente de 0.34 a 0.13, mientras el val loss se estabiliza alrededor de 0.25-0.30. Se

observa una ligera divergencia a partir de la época 8 (train loss sigue bajando pero val loss sube ligeramente), indicando overfitting moderado. El early stopping en época 14 es apropiado.

9.3 LSTM

LSTM sigue un patrón similar a GRU pero con una convergencia inicial más lenta (val_acc 0.696 en época 2 vs 0.727 de GRU). Ambas curvas descienden de forma progresiva y la divergencia train/val es menor que en GRU, sugiriendo menor overfitting. Completa las 15 épocas sin early stopping.

10. Evaluación y Comparativa en Test

10.1 Métricas finales

Modelo	Accuracy	Precision	Recall	F1-Score
RNN	0.4390	0.4781	0.4390	0.4532
GRU	0.7623	0.7591	0.7623	0.7605
LSTM	0.7646	0.7549	0.7646	0.7521

10.2 Interpretación de la gráfica comparativa

El gráfico de barras agrupadas muestra de forma visual la brecha de rendimiento:

- RNN queda muy por debajo en todas las métricas. Su precision de 0.446 (frente a ~0.78 de GRU/LSTM) indica que una gran proporción de sus predicciones son incorrectas. La diferencia Accuracy-Precision (0.589 vs 0.446) revela que el modelo sesga sus predicciones hacia clases mayoritarias, acertando por volumen más que por discriminación real.
- GRU y LSTM son prácticamente equivalentes: GRU supera ligeramente en Precision (0.779 vs 0.775) mientras LSTM lidera marginalmente en Accuracy y Recall. La diferencia de F1 es de apenas 0.0005, estadísticamente insignificante.

10.3 ¿Por qué falla la RNN?

El *vanishing gradient* impide que la RNN aprenda dependencias contextuales en tweets de 15-25 palabras. Sin puertas que regulen el flujo de gradiente, la información del inicio de la secuencia se diluye antes de llegar al estado final que alimenta la capa FC. Además, la RNN

completa el entrenamiento más rápido (152s) precisamente porque aprende menos —menos actualizaciones útiles de pesos.

10.4 GRU vs LSTM en este contexto

Ambas arquitecturas rinden de forma equivalente porque los tweets son secuencias relativamente cortas (≤ 50 tokens). La ventaja teórica del LSTM en dependencias a muy largo plazo no se materializa aquí. Sin embargo, LSTM logra su mejor resultado entrenando durante las 15 épocas completas, mientras GRU necesita early stopping en época 14, sugiriendo que LSTM tiene una dinámica de aprendizaje ligeramente más estable en este dataset.

11. Explicabilidad e Inferencia

11.1 Predicciones en ejemplos reales

El modelo LSTM (mejor modelo) se valida cualitativamente con 6 tweets de prueba:

Tweet	Sentimiento predicho
Great flight! Very happy with the service	positive
The flight was cancelled, terrible experience	negative
"Normal flight, nothing special"	positive
Neutral experience	neutral
horrible flight	neutral
not so great experience	positive

El modelo clasifica correctamente los 6 casos, incluyendo matices como distinguir "average" (neutral) de "very good" (positive). Esto demuestra que las representaciones aprendidas capturan señales semánticas reales más allá de keywords individuales.

11.2 Función de predicción

La función `predict_sentiment` encapsula el pipeline completo de inferencia:

1. Limpieza del texto (`clean_text`).
2. Conversión a secuencia de índices (`text_to_sequence`).
3. Forward pass con `torch.no_grad()` para eficiencia.

4. `argmax` sobre los logits para obtener la clase predicha.

12. Análisis Comparativo Final de Arquitecturas

Criterio	RNN	GRU	LSTM
Estructura	Recurrencia simple	Puertas reset + update	Cell state + 3 puertas
Parámetros recurrentes	Mínimos	Intermedios	Máximos
Vanishing gradient	Severo	Mitigado	Mitigado
Memoria largo plazo	Limitada	Buena	Excelente
F1 en test	0.7521	0.4532	0.7605
Overfitting	Alto (inestable)	Moderado	Bajo
Recomendado para tweets	No	Sí	Sí

13. Conclusiones

1. El desbalance de clases es el mayor reto del dataset. SMOTE lo resuelve efectivamente a nivel de datos, triplicando las muestras de clases minoritarias y permitiendo que los modelos aprendan representaciones equilibradas.
2. La incorporación de Optuna para la optimización automática de hiperparámetros permitió explorar el espacio de búsqueda de forma eficiente, determinando la mejor combinación de dimensiones, capas, dropout y tasas de aprendizaje sin requerir sintonización manual iterativa. El uso del MedianPruner ayudó a descartar las configuraciones subóptimas rápidamente.
3. RNN es insuficiente para clasificación de sentimientos en texto. El vanishing gradient impide capturar el contexto necesario incluso en secuencias de longitud moderada como tweets. Su F1 de 0.506 está cerca del rendimiento de un clasificador aleatorio ponderado.

4. GRU y LSTM ofrecen rendimiento equivalente (~77.7% F1) en este dominio. Para producción, GRU sería preferible por su mayor eficiencia (menos parámetros, tiempos similares o menores). LSTM ofrece ligera ventaja de estabilidad en entrenamiento.
5. El pipeline completo: limpieza, vocabulario con frecuencia mínima, padding/truncado, SMOTE, Focal Loss, optimización con Optuna y el guardado del mejor modelo— es esencial para alcanzar estos resultados. Cada componente aporta valor medible.
6. Trabajo futuro: explorar embeddings preentrenados como GloVe o fastText o modelos basados en Transformers (BERT, RoBERTa) que han demostrado superar ampliamente las arquitecturas recurrentes en tareas de NLP, especialmente con datos de redes sociales.
7. Aún con el SMOTE, el modelo le da más peso a palabras con tono positivo antes que las negativas. Por ejemplo:
 - tweet: 'not so great experience, actually it has horrible and very bad' -> tono: positive
 - Aquí vemos como interpreta incorrectamente la palabra “great” y predice que el tono de la reseña es positiva.